case), and merge the two clusters (`C1` will be in `shard1` and `C2` will be in `shard2`). Optionally we can check that `C1` and `C2` are not one subset of the other, and handle that case differently.

There is one final detail we need to address before wrapping up the discussion on MapReduce DBSCAN: the sharding step. Before continuing reading, try to stop for a minute and think about how this case is different from what we had seen before, and what issues we could face in this step.

Do you see the problem? Deterministically sharding points according to a rectangular grid will not be as cheap as the random sharding we have seen so far! Before running and even configuring the MapReduce job, in fact, to perform this sharding we would need to run a single-thread process creating the grid and assigning each point to a shard, depending on the point's position. If the grid has `m` cells, and the dataset holds `n` points with `d` coordinates each, then this step will require, in the worst case, `O(n*d*m)` comparisons.